Trabalho de Programação em C

Controlando uma Impressora Térmica Elgin

Linguagem C + Impressora Elgin i8 ou i9

Em uma frase: vocês vão completar um programa em C, todo no arquivo **main.c**, que pede login, mostra um menu e manda a impressora **imprimir, apitar, abrir a gaveta** e muito mais.

Nome do grupo: _____

Integrantes: _____

Prazo de entrega: ____ / ____ / _____

1. O que vocês vão fazer

Vocês recebem um programa **quase pronto**. A ligação com a impressora, a configuração da conexão e a abertura da conexão já estão feitas. O trabalho de vocês é **criar o que falta**, copiando e adaptando o que já existe.

Vocês vão usar o que já aprendemos: **variáveis, if/else, laços (for e while), vetores e funções**. **Tudo fica no arquivo main.c**.

Como o C conversa com a impressora

A Elgin criou uma biblioteca (arquivo `.dll`) com os comandos da impressora. No `main.c` já existe uma parte pronta (`carregarDLL()`) que carrega essa biblioteca. Depois disso, mandar um comando é só chamar uma função, como qualquer outra:

```
int retorno = SinalSonoro(2, 50, 5);   /* manda a impressora apitar 2 vezes */
```

Todo comando devolve um **número inteiro**. Se for **0**, deu certo. Se for outro número, deu erro e o programa precisa avisar quem está usando.

> **Atenção:** o comando `ImpressaoTexto` é diferente: ele devolve a quantidade de dados enviados quando funciona. Para ele, só é erro quando o número é **negativo** (retorno < 0).

2. O que já vem pronto no main.c

Parte pronta	Para que serve
typedefs e ponteiros (começo do arquivo)	Descrevem cada comando da DLL. **Não mexam.**
carregarDLL()	Carrega a DLL da raiz do projeto e localiza os 19 comandos. Já é chamada na `main`.
Variáveis **conexaoAberta, tipo, modelo, conexao, parametro**	Guardam os dados da conexão. `conexaoAberta`: 0 = fechada, 1 = aberta.
ASS_QRCODE	Chave do QR Code, usada no cancelamento do XML SAT.
capturarEntrada(mensagem, destino, tamanho)	Mostra a mensagem e guarda em `destino` o **texto** digitado (aceita espaços).
lerInteiro(mensagem)	Mostra a mensagem e devolve um **número**. Se digitar letras, pergunta de novo.
lerArquivo(nome)	Lê um arquivo inteiro e devolve o texto (ou **NULL** se não achar). **Quem usa deve chamar free()**.
configurarConexao()	Pergunta tipo, modelo, porta e parâmetro e guarda nas variáveis.
abrirConexao()	Abre a conexão com a impressora. **É o modelo para copiar**.
exibirMenu() e **main()**	Começos de exemplo: o menu só tem a opção 1 e o laço da `main` ainda não faz nada com a opção.

Arquivos que acompanham o projeto:

Arquivo / pasta	O que é
`main.c`	O programa. **É o único arquivo que vocês editam.**
`E1_Impressora01.dll`	A biblioteca da Elgin. Existem duas versões: **x86** (32 bits) e **x64** (64 bits). A certa precisa ficar na **raiz** do projeto (veja o item 5.3).
`XMLSAT.xml` e `CANC_SAT.xml`	XMLs de teste (dados fictícios) para as opções de impressão de cupom. Ficam na raiz.

3. O que vocês precisam criar

No `main.c` existem funções vazias. Cada uma precisa ser completada, e vocês ainda criam o **login**, o **menu completo** e o **laço da main**.

Função	O que deve fazer	Dica
fazerLogin() (nova)	Pede usuário e senha, **3 tentativas**. Devolve **1** se acertar e **0** se errar as 3. Usuário **admin**, senha **1234**.	Laço **for**. Textos se comparam com `strcmp(a, b) == 0`. Mostre as tentativas que restam.
exibirMenu()	Mostra o menu com **pelo menos 10 opções** + a opção 0 (sair).	Vários `printf`.
fecharConexao()	Fecha a conexão e coloca **conexaoAberta = 0**.	Comando `FechaConexaoImpressora()`. Só se estiver aberta.
imprimirTexto()	Pergunta o texto e a quantidade de cópias e imprime.	`ImpressaoTexto(texto, 1, 4, 0)` e depois `Corte(5)`.
imprimirQRCode()	Pergunta o conteúdo e imprime o QR Code.	`ImpressaoQRCode(dados, 6, 4)` e depois `Corte(5)`.
imprimirCodigoBarras()	Imprime um código de barras.	`ImpressaoCodigoBarras(8, "{A012345678912", 100, 2, 3)` e `Corte(5)`.
imprimirXMLSAT()	Lê o `XMLSAT.xml` e imprime o cupom.	`lerArquivo`, `ImprimeXMLSAT(dadosXML, 0)`, `Corte(5)` e `free`.
imprimirXMLCancelamentoSAT()	Lê o `CANC_SAT.xml` e imprime o cancelamento.	`ImprimeXMLCancelamentoSAT(dadosXML, ASS_QRCODE, 0)` e `free`.
abrirGavetaElgin()	Abre a gaveta de dinheiro.	`AbreGavetaElgin(1, 50, 50)`.
abrirGaveta()	Abre a gaveta de dinheiro (outro comando).	`AbreGaveta(1, 5, 10)`.
emitirSinalSonoro()	Apita a impressora.	`SinalSonoro(4, 50, 5)` (quem quiser pergunta a quantidade).
main()	Faz o login e mantém o menu funcionando até escolher 0.	Laço **while** com **if / else if** para cada opção.

Sugestão de numeração do menu:

Nº	Opção	Nº	Opção
1	Configurar conexão	7	Imprimir XML SAT
2	Abrir conexão	8	Imprimir XML de cancelamento SAT
3	Fechar conexão	9	Abrir gaveta (Elgin)
4	Imprimir texto	10	Abrir gaveta
5	Imprimir QR Code	11	Sinal sonoro
6	Imprimir código de barras	0	Sair

As funções `configurarConexao()` e `abrirConexao()` já estão prontas e valem como as opções 1 e 2.

4. Entendendo o exemplo (leiam com calma!)

Todo comando que usa a impressora segue a **mesma receita**. Vejam como ficaria o `imprimirTexto()`:

```
void imprimirTexto(void) {
    if (!conexaoAberta) {            /* 1) conexão aberta? */
        printf("Conexao fechada. Abra a conexao antes (opcao 2).\n");
        return;                      /*    não está: avisa e SAI da função */
    }

    char texto[200];                 /* 2) só agora pergunta */
    capturarEntrada("Texto a imprimir: ", texto, sizeof(texto));
    int copias = lerInteiro("Quantas copias? ");

    for (int i = 1; i <= copias; i++) {          /* laço: repete a impressão */
        int retorno = ImpressaoTexto(texto, 1, 4, 0);   /* 3) chama o comando */
        if (retorno >= 0) {                      /* 4) confere o retorno */
            printf("Copia %d impressa!\n", i);
        } else {
            printf("Erro ao imprimir. Codigo: %d\n", retorno);
        }
    }
    Corte(5);                        /* corta o papel */
}
```

Regra importante: se a conexão **não está aberta**, a função **não pode perguntar nada** ao usuário (texto, quantidade...). Primeiro confere a conexão, avisa e sai com `return`. Assim ninguém digita tudo à toa. **Todas as funções de impressão precisam começar assim.**

Receita para qualquer função da impressora

1. Se **conexaoAberta** for 0: mostre o aviso e use **return**.
2. Pergunte ao usuário o que for preciso (**capturarEntrada** ou **lerInteiro**).
3. Chame o comando: **int retorno = Comando(...)**.
4. Se **retorno == 0**, mostre sucesso. Senão, mostre o erro e o código.
5. Se a impressão precisa de corte, chame **Corte(5)**.

Exemplo com arquivo XML (lembre do free!)

Quando se usa `lerArquivo`, o programa reserva memória. Depois de usar, é preciso devolver com `free`:

```
void imprimirXMLSAT(void) {
    if (!conexaoAberta) {
        printf("Conexao fechada. Abra a conexao antes (opcao 2).\n");
        return;
    }
    char *dadosXML = lerArquivo("XMLSAT.xml");
    if (dadosXML == NULL) {                  /* arquivo não encontrado */
        printf("Nao foi possivel ler o XMLSAT.xml\n");
        return;
    }
    int retorno = ImprimeXMLSAT(dadosXML, 0);
    Corte(5);
    free(dadosXML);                          /* devolve a memória */

    if (retorno == 0) {
        printf("XML SAT impresso!\n");
    } else {
        printf("Erro ao imprimir XML SAT. Codigo: %d\n", retorno);
    }
}
```

Dica para o login (fazerLogin)

```
int fazerLogin(void) {
    char usuario[50], senha[50];
    for (int tentativa = 1; tentativa <= 3; tentativa++) {
        /* pedir usuário e senha com capturarEntrada(...) */
        /* se acertou: if (strcmp(usuario, "admin") == 0 && strcmp(senha, "1234") == 0) return 1; */
        /* se errou: mostrar quantas tentativas restam (3 - tentativa) */
    }
    /* chegou aqui? errou as 3 vezes: avisar e return 0; */
    return 0;
}
```

Dica para a main

```
if (!fazerLogin()) {             /* depois do carregarDLL() */
    printf("Acesso negado.\n");
    FreeLibrary(dll);
    return 1;
}
int opcao = -1;
while (opcao != 0) {
    exibirMenu();
    opcao = lerInteiro("\nDigite a opcao desejada: ");
    if (opcao == 1) {
        configurarConexao();
    } else if (opcao == 2) {
        abrirConexao();
    }  /* ... um else if para cada opção ... */
    else if (opcao == 0) {
        fecharConexao();
    } else {
        printf("Opcao invalida.\n");
    }
}
```

Cuidados em C: (1) uma função precisa estar escrita **antes** de ser usada (por isso `fazerLogin` vem antes da `main`); (2) textos comparam-se com `strcmp`, nunca com `==`; (3) nos `printf` evitem acentos, para não aparecerem símbolos estranhos no console do Windows.

5. Passo a passo para trabalhar

5.1 Entendendo a DLL em C

No Java existe uma biblioteca para ligar a DLL. No C, quem faz isso é o próprio Windows, com duas funções que já estão dentro do `carregarDLL()`. Vocês **não precisam mexer**, mas vale entender:

```
dll = LoadLibraryA("E1_Impressora01.dll");      /* abre a DLL (procura na pasta do programa) */
SinalSonoro = GetProcAddress(dll, "SinalSonoro"); /* acha o comando pelo nome */
int retorno = SinalSonoro(2, 50, 5);             /* agora é só chamar */
```

Por isso, 3 coisas precisam estar certas: (1) a `E1_Impressora01.dll` na **raiz** do projeto; (2) o programa rodando a partir da **pasta do projeto** (os XMLs também são procurados lá); (3) a DLL com os **mesmos bits (32 ou 64)** do compilador.

5.2 Montar o projeto

- Crie uma pasta para o projeto e coloque nela o `main.c`, a `E1_Impressora01.dll` certa (item 5.3) e os dois XMLs.
- **Code::Blocks:** *File → New → Project → Console application → C*. Troque o conteúdo do `main.c` pelo código do professor e copie a DLL e os XMLs para a **pasta do projeto** (a mesma do arquivo .cbp).
- No Code::Blocks, confira em **Project → Properties → Build targets** o campo *Execution working dir*: deve ser a pasta do projeto.
- **Pelo terminal** (compilador gcc): entre na pasta do projeto e rode os dois comandos abaixo.
- Ligue a impressora e conecte o cabo USB antes de testar.

```
gcc main.c -o impressora.exe
impressora.exe
```

5.3 DLL de 32 ou 64 bits

A DLL e o compilador precisam ser do **mesmo tipo**. Para descobrir o do seu compilador, rode no terminal:

```
gcc -dumpmachine
```

Resultado	Seu compilador é	Use a DLL da pasta
`x86_64-w64-mingw32`	64 bits	**x64**
`mingw32` ou `i686-w64-mingw32`	32 bits (o MinGW do Code::Blocks costuma ser assim)	**x86**

5.4 Ordem sugerida

- **1º** Compile e rode o programa. Se aparecer o menu, a DLL carregou.
- **2º** Faça **exibirMenu()** completo, **fazerLogin()** e o laço da **main()**.

- **3º** Teste as opções 1 e 2 (**configurarConexao** e **abrirConexao**, já prontas) com a impressora ligada.
- **4º** Faça o **imprimirTexto()** seguindo o exemplo do item 4. Se imprimiu, a comunicação está funcionando!
- **5º** Faça **uma função por vez**, copiando a anterior, e teste cada uma na impressora.
- **6º** Faça o **fecharConexao()** e confira se ele é chamado ao sair (opção 0).
- **7º** Teste tudo de novo e preencha o README.

5.5 Se der erro

O que apareceu	O que provavelmente é
Nao foi possivel carregar E1_Impressora01.dll (erro 126)	A DLL não está na pasta de onde o programa é executado (raiz do projeto / Execution working dir).
... (erro 193)	DLL com bits diferentes do compilador (item 5.3).
Comando '...' nao encontrado na DLL	DLL errada ou de outra versão. Use a que o professor entregou.
Erro ao abrir a conexão, código **-9999** ou negativo	Impressora desligada, cabo solto, ou modelo/porta errados na configuração (opção 1).
Menu aparece mas a opção nada faz	Faltou o `else if` dessa opção na `main`.
Nao foi possivel ler o XML	O arquivo XML não está na pasta de execução do programa.

6. O que o programa precisa ter

- **Login** com no máximo 3 tentativas e aviso das tentativas restantes.
- **Menu** com pelo menos **10 opções** e a opção 0 para sair, repetindo até o usuário sair.
- **Cada opção** chama uma função diferente.
- **Nenhuma pergunta de impressão** quando a conexão está fechada.
- **Todo comando da impressora** tem o retorno conferido, com mensagem de sucesso ou erro.
- **Memória liberada:** todo `lerArquivo` tem seu `free`.
- O programa **não quebra** se digitarem letras onde se espera número (o `lerInteiro` já cuida disso).

7. Consulta rápida: comandos da impressora

Resumo da documentação oficial da Elgin.

Comando	Parâmetros
AbreConexaoImpressora(tipo, modelo, conexao, parametro)	**tipo:** 1 = USB, 2 = Serial, 3 = Rede (IP), 4 = Bluetooth. **modelo:** "i8" ou "i9". **conexao:** "USB" para USB. **parametro:** 0 para USB. Retorna 0 se der certo.
FechaConexaoImpressora()	Sem parâmetros. Retorna 0 se der certo.
ImpressaoTexto(dados, posicao, estilo, tamanho)	**posicao:** 0 = esquerda, 1 = centro, 2 = direita. **estilo:** 0 = fonte A, 2 = sublinhado, 8 = negrito (somam). **tamanho:** 0 = normal; 1 a 7 = altura maior. **Erro = retorno negativo.**
Corte(avanco)	Avança o papel e corta. Ex.: 5.
ImpressaoQRCode(dados, tamanho, nivelCorrecao)	**tamanho:** 1 a 6. **nivelCorrecao:** 1 a 4.
ImpressaoCodigoBarras(tipo, dados, altura, largura, HRI)	**tipo:** 2 = EAN13, 3 = EAN8, 4 = CODE 39, 8 = CODE 128. **altura:** 1 a 255. **largura:** 1 a 6. **HRI:** 1 = número acima, 2 = abaixo, 3 = os dois, 4 = não imprime. Em CODE 128 o texto começa com {A.
AvancaPapel(linhas)	Quantas linhas avançar.
StatusImpressora(param)	1 = gaveta, 2 = tampa, 3 = papel, 5 = geral.
AbreGavetaElgin(pino, ti, tf)	**pino:** 0 ou 1. **ti** e **tf:** tempos. Ex.: 1, 50, 50.
AbreGaveta(pino, ti, tf)	**pino:** 0 ou 1. **ti** e **tf:** 1 a 255. Ex.: 1, 5, 10.
SinalSonoro(qtd, tempoInicio, tempoFim)	**qtd:** 1 a 63. **tempoInicio:** ex. 50. **tempoFim:** 1 a 25, ex. 5.
ImprimeXMLSAT(dados, param)	**dados:** o texto do XML. **param:** 0 = padrão.
ImprimeXMLCancelamentoSAT(dados, assQRCode, param)	**dados:** o texto do XML de cancelamento. **assQRCode:** use a variável ASS_QRCODE. **param:** 0.

Valores para os testes: tipo = 1 (USB), modelo = "i9" (ou "i8"), conexao = "USB", parametro = 0. Documentação completa: `https://elgindevelopercommunity.github.io/group___m1.html`

8. O que entregar

- O arquivo **main.c** completo e funcionando na impressora.
- O projeto publicado no **GitHub**, com pelo menos **3 commits** com mensagens claras (exemplo: *"Adiciona o login"*). Não envie o `.exe` gerado.
- Um **README.md** com: nome do projeto, o que o programa faz, como compilar e executar (e qual DLL usar, 32 ou 64 bits), login de teste, lista das opções do menu e os autores.
- O **link do repositório** enviado ao professor no prazo combinado.
- O programa **funcionando na impressora**, mostrado em sala.

Comandos básicos do Git

```
git init                      # cria o repositório (uma vez só)
git add .                     # prepara os arquivos alterados
git commit -m "Mensagem"      # grava uma versão com uma mensagem
git remote add origin LINK    # liga ao repositório do GitHub (uma vez só)
git push -u origin main       # envia para o GitHub
```

9. Avaliação (10 pontos)

O que será avaliado	Pontos
Login com 3 tentativas funcionando	1,5
Menu com 10 ou mais opções, em laço, com opção de sair	1,5
Funções de comunicação criadas e funcionando na impressora	3,0
Impressão bloqueada (sem perguntas) quando a conexão está fechada	1,0
Retorno de cada comando conferido, com mensagens claras (e free nos XMLs)	1,0
Repositório no GitHub e README	1,0
Código organizado: nomes claros e comentários	1,0
Total	**10,0**

Bom trabalho!